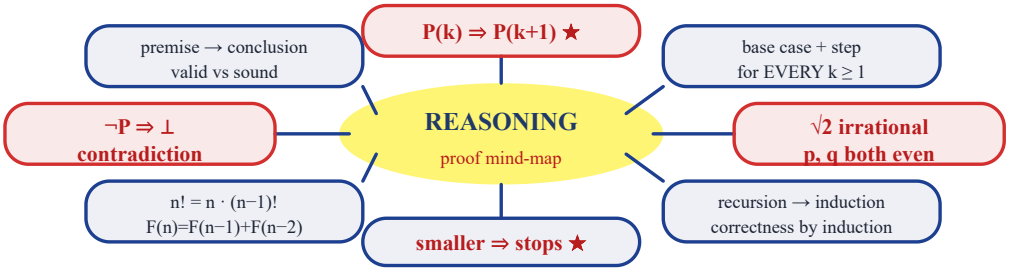
Mathematical Reasoning

Induction · Contradiction · Recursion — the three proof machines of the syllabus: exam-ready in 13 sheets



↑ the whole topic on one page: red = high-yield, blue = definitions

Syllabus coverage

"Discrete Mathematics, Theory of Computation and Basic Graph Theory: Set theory; Relations; Functions; Graph theory; Propositional calculus and predicate calculus; **Mathematical reasoning: induction, contradiction and recursion**; Counting; …" Sets and functions are in the first lesson, graph theory and the two calculi in the previous one.

High-yield list

① Induction = base case P(1) + step **P(k) ⇒ P(k+1)**

② the step must hold for **every** k ≥ 1, and **forward**

③ 1+2+…+n = **n(n+1)/2**; 1+2+2²+…+2ⁿ = **$2^{n+1} - 1$**

④ Contradiction: assume **the negation** of P, derive ⊥

⑤ √2 irrational: $p^2 = 2q^2$ ⇒ p, q both even ⇒ gcd ≠ 1

⑥ Contrapositive ¬Q → ¬P is **equivalent**; converse is not

⑦ n! = 1 (n = 0) else n·(n−1)!; F(n) = F(n−1)+F(n−2)

⑧ Recursion must **strictly decrease** or it never ends

⑨ naive fib(5) = **15** calls ≈ 1.618^n; factorial = **6**, O(n) · ⑩ induction ⇒ **tree: n ⇒ n − 1 edges**

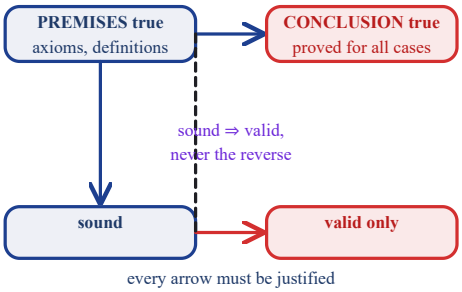
Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ What a proof is … p. 2
✓ Induction: the two parts … p. 3
✓ Worked proof: Σ 1…n … p. 4
✓ Worked proof: Σ powers of 2 … p. 5
✓ **Contradiction** … p. 6–7 · **Contrapositive** … p. 8 · **Recursion** … p. 9–10 · the three compared … p. 11 · Night-before p. 12 · Practice p. 13

1 What a Proof Is

Definition: **Mathematical reasoning** is the logical process of deducing conclusions from given statements or **premises**. A **proof** is an argument whose every inference is valid; it is **valid** when the conclusion cannot fail however the premises turn out, and **sound** when the premises were also true.
A proof must cover every admissible case, not the ones you tried.



sound needs true premises **and** valid steps;
valid needs only the steps

Method	Shape of the argument	Best for
Direct proof	P follows from the premises by known rules	most theorems
Contradiction	**assume ¬P**, derive the impossible	"no such thing exists"
Contrapositive	P → Q proved as ¬Q → ¬P — **equivalent**	implications denying a hypothesis
Induction	P(1) holds, and P(k) ⇒ P(k+1) for all k	statements about $\mathbb{N}$
Counterexample	one failing case kills a universal claim	disproving, never proving

Exam trap: A **counterexample disproves** but never proves, and a **single true instance** never proves a universal claim. "I checked n = 1 … 10 and it held" is **not** a proof about all n — that gap is exactly what induction was invented to close.

What counts as a premise: **definitions**, which you may unfold inside the proof · **axioms** of the branch in hand · **lemmas** you have already proved · the **hypotheses** of the statement itself. What does **not** count: "it is well known", "the diagram makes it clear", "it checks out for small cases".

Memory trick — naming the method: Read the **shape** of the question, not the topic. "For every n ≥ 1 …" → **induction**. "There is no …" or "show that not …" → **contradiction**. "If … then …" → **contrapositive**. "Is there any …?" → hunt for a **counterexample** to show there is not.

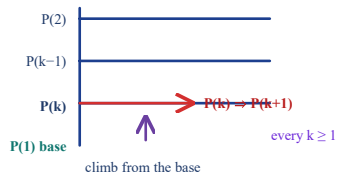
2 Mathematical Induction: the Two Parts

Definition: **Mathematical induction** proves a statement P(n) about the natural numbers from a single **principle of mathematical induction**: if P(1) is true **and** P(k) ⇒ P(k+1) for **every** integer k ≥ 1, then P(n) is true for **every** n ≥ 1. Both halves are compulsory — the **base case** alone proves nothing, and the **inductive step** alone starts from nothing.

Part	What you must write	If you omit it
1. Base case (n = 0 or 1)	"For n = 1, P(1) is **true**." Check it numerically.	no bottom rung
2. Inductive step	"Let **k ≥ 1** be arbitrary and **assume P(k)**; show **P(k+1)**."	one case proved
3. Conclusion	"By the principle of mathematical induction, P(n) holds for all n ≥ 1."	no principle named

Memory trick: "**B**ase **B**uilds the first brick; the **S**tep carries the idea one rung further, and it must carry it **every** time." The step is a **rule**, not a **sample** — write "suppose P(k) is true for an **arbitrary** k ≥ 1", never "let k = 3".

Statement P(n) you may be asked to prove	Base case	Shape of the step
$1 + 2 + \dots + n = n(n+1)/2$	**n = 1**	add the term k+1 to both sides
$1 + 2 + 2^2 + \dots + 2^n = 2^{n+1} - 1$	**n = 0**	double the whole left-hand side
$n^2 \geq n$ for every n ≥ 1	**n = 1**	$(k+1)^2 = k^2 + 2k + 1 \geq k^2 + k$
n(n+1) is even for every n ≥ 1	**n = 1**	(k+1)(k+2) = k(k+1) + 2(k+1)
n ∣ n! for every n ≥ 1	**n = 1**	(k+1)! = (k+1) · k!



each rung is proved **using the rung below it** — for an arbitrary k, not a chosen one

Reading the base case off the statement: The first value of n is fixed by the formula, not by taste. 1 + 2 + … + n starts at **n = 1**; $1 + 2 + 2^2 + \dots + 2^n$ starts at **n = 0**; n! uses **n = 0**. If the claim says "for all n ≥ 0" the base case **must** be n = 0 — starting at n = 1 leaves n = 0 unproved.

3 Worked Proof: 1 + 2 + 3 + … + n = n(n+1)/2

Worked example ✍ full proof: **Let P(n):** 1 + 2 + 3 + … + n = n(n+1)/2. **(1) Base case (n = 1).** LHS = 1, RHS = 1(1+1)/2 = 1, so **P(1) is true**. **(2) Inductive hypothesis.** Let **k ≥ 1** be arbitrary and **suppose P(k)**: 1 + 2 + … + k = k(k+1)/2. **(3) Inductive step.** Show P(k+1): 1 + 2 + … + k + (k+1) = k(k+1)/2 + (k+1) (by the hypothesis) = (k+1)[k/2 + 1] = (k+1)(k + 2)/2 = (k+1)((k+1) + 1)/2 — exactly the right-hand side of P(k+1), so **P(k+1) is true**. **(4) Conclusion.** P(1) holds and P(k) ⇒ P(k+1) for every k ≥ 1, so by the principle of mathematical induction P(n) is true for every n ≥ 1. ■

Exam trap — the single highest-yield mistake in this lesson: The inductive step must show **P(k) ⇒ P(k+1) for EVERY integer k ≥ 1**. Three failures score nothing. (i) Verifying the step at **one particular value** of k, say k = 3 — that is a sample, not a proof. (ii) Writing the **reverse** implication P(k+1) ⇒ P(k), a completely different statement. (iii) Dropping the words "**assume P(k)**" and simply asserting the result for k+1. If a proof can be summarised as "it held for k = 1, 2, 3, 4, 5", it is not induction.

How to write the step so the examiner is satisfied: (i) name the statement "P(n): …"; (ii) base case, numerically; (iii) "assume P(k) for an arbitrary k ≥ 1"; (iv) compute the k+1 case **starting from the hypothesis** and mark where it was used; (v) finish with "by the principle of mathematical induction". Marks are awarded per line — do not skip (v).

Worked example ✍ why the hypothesis matters: The step for this proof is a one-line substitution, but only **because** "1 + 2 + … + k" was replaced by its closed form k(k+1)/2. Write the line without that substitution and you have merely restated the claim for k+1 — which is **exactly what induction asks you not to do**. The hypothesis is the engine; a step that never calls the engine proves nothing.

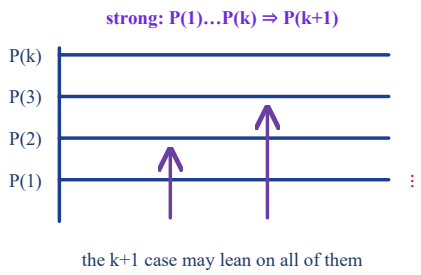
Worked example ✍ two lines that look like proofs and are not:
(i) "It holds for n = 1, 2, 3, so it holds for all n." This tests **one** value of k in the step — a sample, not a proof. (ii) "P(k+1) ⇒ P(k), hence by induction." This runs the ladder **backwards**: even if the reverse implication is true, it says that knowing a statement at k+1 tells you about k, which never gets you from the base case to the top. **Only P(k) ⇒ P(k+1) climbs.**

When induction beats direct algebra: A claim that carries an **n** on both sides — n!, Fibonacci, running sums, divisibility — cannot be rearranged usefully, and the recursive structure of the statement matches the shape of the step exactly. For a claim like "$n^2 + n$ is always even", a one-line factorisation n(n+1) is shorter and just as correct; the examiner accepts either. Reach for induction when the statement is **built from the previous case**.

4 Second Worked Proof: 1 + 2 + 2² + … + 2ⁿ = 2ⁿ⁺¹ − 1

Worked example ✍ the standard written answer: **Let P(n):** 1 + 2 + 2² + … + $2^n = 2^{n+1} - 1$, for every integer n ≥ 0. **(1) Base case (n = 0).** LHS = $2^0 = 1$, RHS = $2^1 - 1 = 1$, so **P(0) is true**. **(2) Inductive hypothesis.** Let k ≥ 0 be arbitrary and assume P(k): 1 + 2 + 2² + … + $2^k = 2^{k+1} - 1$. **(3) Inductive step.** We show P(k+1): 1 + 2 + … + $2^k + 2^{k+1} = (2^{k+1} - 1) + 2^{k+1}$ (by the hypothesis) = 2 · $2^{k+1} - 1 = 2^{k+2} - 1$ = 2^(k+1)+1 − 1 — this is exactly P(k+1), so **P(k+1) is true**. **(4) Conclusion.** P(0) holds and P(k) ⇒ P(k+1) for every k ≥ 0, so by the principle of mathematical induction P(n) is true for all integers n ≥ 0. ■

Definition — strong (complete) induction: To prove P(n) for all n ≥ 1 it is enough to show: **(a)** P(1) is true, and **(b)** for every k ≥ 1, if **P(1), P(2), …, P(k) are all true, then P(k+1) is true**.
Strong induction is ordinary induction with a stronger hypothesis, and the two are equivalent — nothing can be proved by one that the other cannot. It is used exactly when the k+1 case genuinely needs several earlier cases at once.



strong induction: several rungs of the ladder may be used at once

Worked example ✍ strong induction: Prove every integer n ≥ 2 has a prime divisor. Base n = 2: 2 is prime, so 2 | 2. Step: suppose every integer in 2 … k has a prime divisor and consider k+1. If k+1 is prime the claim is immediate; if it is composite it has a divisor d with 1 < d < k+1, and by the strong hypothesis d has a prime divisor p, which then also divides k+1.
Hence every n ≥ 2 has a prime divisor.

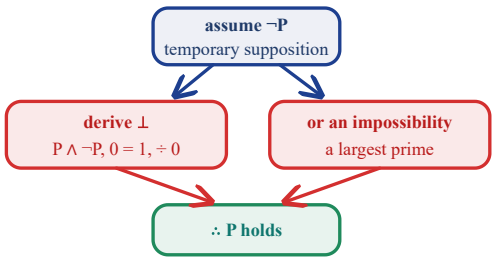
Exam trap: (i) Induction proves statements about ℕ only. (ii) The step must read "**for every k ≥ 1**"; a candidate who writes "for some k" has not proved it. (iii) The hypothesis must actually be **used** — if the k+1 line follows without it, the argument was circular. (iv) If the k+1 case needs two earlier cases, either use **strong** induction or the statement is false as written.

Worked example ✍ a third routine proof: Prove $1 + 1/2 + 1/2^2 + \dots + 1/2^n = 2 - 1/2^n$ for n ≥ 0. Base n = 0: LHS = 1, RHS = 2 − 1 = 1 ✓. Step: assume the sum to k is $2 - 1/2^k$; then the sum to k+1 is $(2 - 1/2^k) + 1/2^{k+1} = 2 - 2/2^{k+1} + 1/2^{k+1} = \mathbf{2 - 1/2^{k+1}}$ ✓ — exactly P(k+1). Same three lines, three different statements.

When induction must NOT be used: Induction proves P(n) for **natural** n and nothing else. A claim quantified over the reals ("the square of an irrational is irrational" — false) cannot be settled by it, and a claim about **infinitely many** real values is not the kind of statement the principle speaks to. Equally, a claim that fails for one natural n is **disproved**, not proved: no induction can rescue a false statement.

5 Proof by Contradiction

Definition: Also called **reductio ad absurdum**. To prove P: **assume ¬P** — the assumption is a temporary **supposition**, not a claim — and from it derive something impossible: a statement together with its negation, 0 = 1, a division by zero, an object with contradictory properties. That impossibility is written ⊥. Since ¬P leads to absurdity, **P must be true**.



anything impossible will do — **one** contradiction finishes the proof

What counts as a contradiction ⊥	Classic use
a statement together with its **negation**	"p even **and** odd"
0 = 1, or any false arithmetic identity	√2 rational
division by zero	1 has no multiplicative inverse
an object with impossible properties	"the largest prime", "a largest integer"
a set both **non-empty and empty**	no bijection $\mathbb{N} \to \mathbb{N} \setminus \mathbb{N}$

Memory trick: "Astract the **p**ositive: assume the **n**egation, **p**ush it to the **absurd**, and the original falls out." The tell-tale of the method is the word **"suppose"** / "assume" followed by something you immediately start attacking.

The recipe, in the four lines an examiner expects: **(1)** State the claim P explicitly. **(2)** Write "suppose, for contradiction, that ¬P", and keep the negation as a **supposition**. **(3)** Derive a chain of consequences and push it to an impossibility ⊥. **(4)** Close with "the assumption is false, therefore P" — the contradiction is about the **assumption**, never about P.

Exam trap: "A proof by contradiction begins by assuming" — the answer is **the negation of the statement to be proved**. Not "the statement is true" (that is circular reasoning), not "the converse", not "the base case P(1)". And the derivation must end in something **impossible**, not merely **unlikely**: "the evidence points the other way" is no contradiction at all.

When contradiction is the wrong tool: If a claim fails for **one** value, do not build an absurd assumption — exhibit the **counterexample** and stop. And a claim about **real** numbers cannot be settled by induction: induction is a theorem about $\mathbb{N}$ only.

6 Contradiction in Action

Worked example ✍️ the canonical proof that √2 is irrational:

1. **Suppose, for contradiction, that √2 is rational.** Then √2 = p/q with integers p, q, q ≠ 0 and **gcd(p, q) = 1** in lowest terms.
2. Square both sides: $2 = p^2/q^2$, hence $\mathbf{p^2 = 2q^2}$.
3. p^2 is even, so **p is even** — an odd p has an odd p^2 — write p = 2k.
4. Substitute: $(2k)^2 = 2q^2 \Rightarrow 4k^2 = 2q^2 \Rightarrow \mathbf{q^2 = 2k^2}$.
5. q^2 is even, so **q is even**.

6. Then 2 divides both p and q, so **gcd(p, q) ≥ 2** — contradicting gcd(p, q) = 1. ⊥
7. Therefore the assumption was false and **√2 is irrational**. ■

Worked example ✍ infinite descent: **Claim:** no sequence of positive integers can descend for ever. **Suppose** one does: $a_1 > a_2 > a_3 > \ldots$, all positive integers. By **well-ordering** $\{a_1, a_2, \ldots\}$ has a least member a_m. But $a_{m+1} < a_m$ is *also* in the set and smaller — contradicting leastness.
Contradiction ⇒ every set of positive integers has a least member
. This is **infinite descent**, the engine behind "√2 is irrational".

Worked example ✍ a family that fails: **Claim:** f(n) = n² + n + 41 is prime for every natural n. **Suppose** it is prime for all n. Take **n = 41**: f(41) = 1681 + 41 + 41 = **1763 = 41 × 43**, which is not prime.
A single counterexample destroys the universal claim — contradiction and counterexample agree here, which is why one bad case is the cheapest refutation a candidate can offer. (f(n) is prime for n = 0 … 39, which is what makes it tempting.)

Exam trap: In the √2 proof the **lowest-terms** hypothesis gcd(p, q) = 1 is what makes the final contradiction bite; without it the derivation is circular. Also "p² is even ⇒ p is even" is a **lemma** that must be stated: what is immediate from p² = 2q² is only that p² is even, and the step to p is the **contrapositive** of "p odd ⇒ p² odd".

Contradiction in the computer-science syllabus: The same shape proves that no total computable function decides a Turing machine's halting behaviour — assume the decider D exists, ask D about D, and D must be wrong about itself. Expect the written papers to ask you to **name the assumption** and **point at the contradiction**; both lines carry marks.

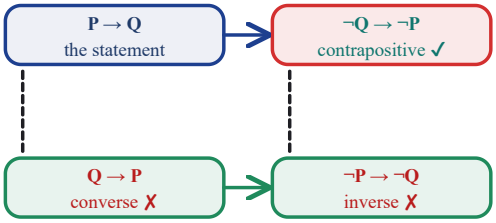
Worked example ✍ there is no largest prime: **Claim:** the primes are unbounded. **Suppose** $p_1, \ldots, p_m$ is a list of **all** the primes and let $P = p_1 p_2 \ldots p_m + 1$. Every p_i divides $P - 1$, so no p_i divides P.
P > 1 therefore has a prime divisor that is not on the list — contradiction.
So there is a prime above any given prime, and hence **no largest prime**.

Exam trap — assume the FULL negation: For a compound claim "every A is a B", the negation is **"there is an A that is not a B"** — not "not every A is a B" left hanging, and not "some A is not a C". Pushing on a partial negation is the commonest way a contradiction proof loses its last mark.

7 Contradiction vs Contrapositive

Definition: To prove **P → Q** by the **contrapositive** you prove **¬Q → ¬P**. That is a different argument with the same power, because P → Q is **logically equivalent to ¬Q → ¬P**. Contrapositive, by contrast, assumes **¬(P → Q) = P ∧ ¬Q** — the hypothesis **true** and the conclusion **false** at once. The **converse** Q → P and the **inverse** ¬P → ¬Q are **not** equivalent to P → Q, though they are equivalent to each other.

Form	Called	Equivalent to P → Q?	Example with P = "n is even", Q = "n² is even"
P → Q	the statement itself	—	true
¬Q → ¬P	**contrapositive**	**yes — always**	"n² odd ⇒ n odd" — true
Q → P	converse	**no**	"n² even ⇒ n even" — false, n = 3
¬P → ¬Q	inverse	**no** (but converse ≡ inverse)	"n odd ⇒ n² odd" — true, yet not equivalent



only the top-right form always carries the top-left form

Exam trap: The two mistakes that cost the mark: (i) calling **¬Q → ¬P** the "converse" — it is the **contrapositive**; (ii) believing that a true inverse makes P → Q true. If both P and Q happen to be true the whole family is true, but equivalence is a statement about **all** cases, not one.

IT link — the same machinery, next chapter: Contradiction settles an **undecidability**: the **halting problem** is proved undecidable by assuming a decider D exists and asking D about D itself — D must accept exactly the machines that do **not** halt on D, a contradiction, so no D exists. Ordinary **induction** settles a **theorem**: a tree with n vertices has exactly **n − 1** edges — proved on n, using the leaf lemma (a longest path ends in a leaf) and deleting it.
One method proves a thing exists; the other proves nothing can.

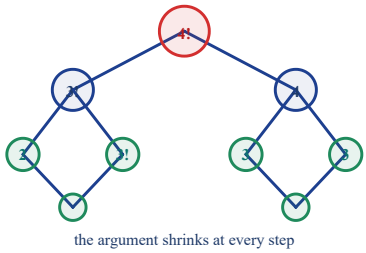
Worked example ✍ the contrapositive doing the work:
Prove: if n² is divisible by 4 then n is even.
Contrapositive: suppose n² is **not** divisible by 4; then n is odd, n = 2k+1, so n² = 4k² + 4k + 1, which is 1 more than a multiple of 4 — **not** divisible by 4.
The impossible remainder 1 is the ⊥.

The four forms on one line: **P → Q** the claim · **¬Q → ¬P** the contrapositive, always equivalent · **Q → P** the converse, may fail · **¬P → ¬Q** the inverse, may fail but equals the converse. Test: find **one** assignment where the two sides differ.

8 Recursion

Definition: A **recursive** definition defines an object **in terms of smaller instances of itself**; a **recurrence relation** does the same for sequences. Every recursive definition has exactly two parts: a set of **base cases** where the recursion stops, and a **recursive step** whose argument is **strictly closer to the base case**. A function that calls itself on an argument which does not decrease is not a recursion — it is an **infinite loop** that overflows the stack.

Object	Base case(s)	Recursive step
Factorial n!	n! = 1 if n = 0	n! = n × (n−1)! for n > 0
Fibonacci F(n)	F(0) = 0, F(1) = 1	F(n) = F(n−1) + F(n−2) for n > 1
Power a^n	$a^0 = 1$	$a^n = a \times a^{n-1}$
Sum S(n)	S(0) = 0	S(n) = S(n−1) + n
Euclid gcd(a, b)	gcd(a, 0) = a	gcd(a, b) = gcd(b, a mod b)



unrolling **factorial(4)**: every call moves to a smaller n and reaches n = 0

```
function factorial(n):
    if n == 0:            # base case — the recursion stops here
        return 1
    else:                 # step: n - 1 is strictly smaller than n
        return n * factorial(n - 1)
```

```
F(0)=0  F(1)=1  F(2)=1  F(3)=2  F(4)=3  F(5)=5  F(6)=8  F(7)=13  F(8)=21
```

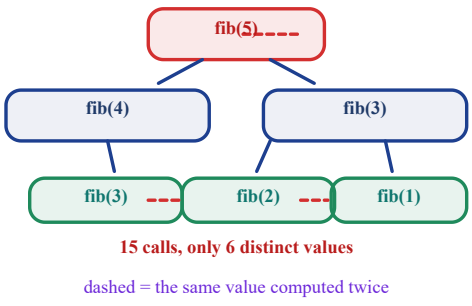
Exam trap: (i) **0! = 1** — the factorial base case is n = 0, not n = 1; since n! = 1 for both n = 0 and n = 1, checking only n = 1 does not verify the base case. (ii) Recursion and iteration compute the **same** result; neither is "the correct one", but naive recursion may be **exponentially slower** — see the next sheet. (iii) A **recurrence relation** (an n → n+1 rule plus starting values) is **not** the same thing as a recursive algorithm; solving one is usually the other run backwards.

Fibonacci indexing: The same sequence is written **F(0) = 0, F(1) = 1** or **F(1) = 1, F(2) = 1** — one index apart. Before answering an MCQ, check which convention the question uses; a question that says "F(1) = 1, F(2) = 1" is asking for the 7th value as **13**, not 8.

Why "smaller" is the whole condition: Termination rests on a **well-founded measure**: some quantity — the argument n, a list size, a tree depth — must **strictly decrease** at every call and must have a **lower bound**. ℕ has none, so repeated decrease forces arrival at 0 or 1. That is why the factorial step uses **n − 1** and not **n + 1**.

9 Recursion ↔ Induction, and What It Costs

Why the two are one idea: A recursive definition **supplies** the base cases and the step; a proof by induction **verifies** exactly those. So **to prove a recursive function correct, run induction on its argument**: base case — evaluate it at the base input; step — assume correctness for n, then the recursive call on **n−1** is covered by the hypothesis, and finish the algebra. This is the standard template for **algorithm correctness proofs**.



naive recursion re-computes the same sub-problems: exponential in n

Worked example ✍️ the template, applied: **Claim:** the function above returns n! for every $n \geq 0$. **Base:** factorial(0) returns $1 = 0!$. **Step:** assume factorial(k) returns k! for some $k \geq 0$; then factorial(k+1) returns $(k+1) \times \text{factorial}(k) = (k+1) \times k! = (k+1)!$. Hence by induction the function is correct for all $n \geq 0$.
Correctness proved; cost still open.

Naive algorithm	Calls made	Cost
factorial(5)	5!, 4!, 3!, 2!, 1!, 0! = **6**	**O(n)** time, **O(n)** stack
fib(5), naive	$T(n) = T(n-1)+T(n-2)+1 \Rightarrow T(n) =$ **$2F(n+1) - 1$**; $T(5) =$ **15**	**$O(\varphi^n)$**, $\varphi = (1+\sqrt{5})/2 \approx 1.618$
fib(5), memoised	each F(i) computed once: **n** calls	**O(n)** time

Exam trap: "factorial is recursive so it must be slow" — **false**: it makes only n + 1 calls, i.e. **O(n)**, the same as a loop. The blow-up is **Fibonacci**, whose call tree branches into two. A **tail** call (the recursive call is the last operation) can be rewritten as a loop and then costs O(1) stack.

Where this earns marks: These four lines — **correctness by induction**, **complexity of the call tree**, **program verification** and **theorem proving** — are the applications the syllabus names. Named systems built on them: **loop invariants** plus induction give total correctness of loops; **structural induction** on trees and lists proves compiler and datatype theorems; **diagonalisation** is contradiction applied to computability.

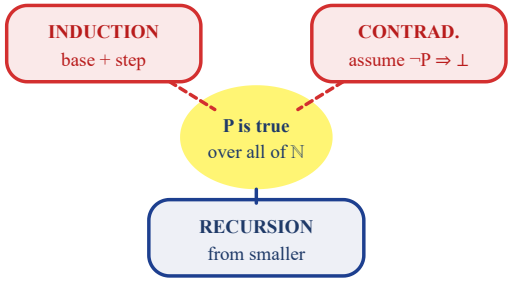
10 Which Method Do I Use?

Method	Key idea	You must supply	Fails when
Induction	prove true for a base case, then **step by step**	P(b) true, and P(k) ⇒ P(k+1) for **every** k	the claim is about $\mathbb{R}$, or the step needs several earlier cases
Contradiction	**assume false**, derive an impossible result	¬P, then a genuine ⊥ ($P \wedge \neg P$, $0 = 1$, $\div 0$)	you assume P itself, or derive something merely **unlikely**
Recursion	**define in terms of smaller instances**	base cases, plus a strictly decreasing step	the argument does not shrink — the recursion never terminates

The three in one line each: **Induction** — "it is true once, and if it is true at k then it is true at k+1", so the truth spreads over all of $\mathbb{N}$. **Contradiction** — "if it were false we would get an impossibility", so it cannot be false. **Recursion** — "define it from the smaller ones", which is the **same information** the induction proof then verifies. **Induction and recursion are two halves of one object.**

Applications — the four the syllabus names: **Algorithm correctness proofs** (induction on the loop counter or the recursion depth) · **Complexity analysis** (solve a recurrence by expansion or the master method) · **Program verification** (pre-condition, post-condition and invariants proved by induction) · **Mathematical theorem proving** (in a formal system a theorem is provable only if it follows from axioms by the inference rules of the previous lesson).

Exam trap: The three are **not** three varieties of one thing: induction and contradiction are **proof techniques** — they argue about truth — while recursion is a **definition and a computational technique**. "Proving by recursion" is only meaningful as "proving the recursive program correct", and that proof is an **induction**. A claim with an easy **counterexample** is never proved by contradiction; it is simply **disproved**.



two proof techniques and one definition technique — they answer different questions

Night-Before-the-Exam Sheet

Fact sheet:

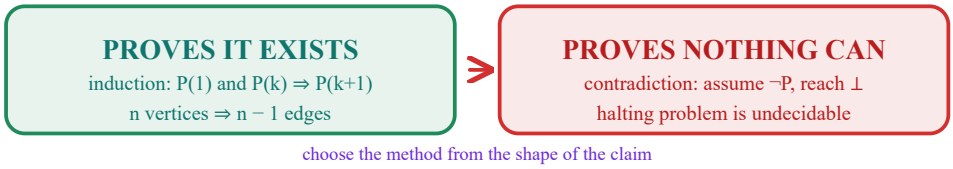
Proof = **valid** + true premises = **sound**	Sound ⇒ valid, never the reverse
Induction: **base case** P(1)	Induction: **step** P(k) ⇒ P(k+1)
Step must hold for **every** k ≥ 1	P(k+1) ⇒ P(k) is not it
Σ 1…n = **n(n+1)/2**	Σ $2^0 \ldots 2^n$ = **$2^{n+1} - 1$**
Strong induction uses **P(1)…P(k)**	Strong ≡ ordinary induction
Contradiction: assume **¬P**	⊥ = P∧¬P, 0 = 1, ÷ 0
√2 irrational: p, q both even	Then gcd(p, q) ≥ 2 ≠ 1
Contrapositive **¬Q → ¬P** ≡ P → Q	Converse Q → P and inverse are not
n! = 1 (n=0), else n·(n−1)!	F(0)=0, F(1)=1, else F(n−1)+F(n−2)
Recursion must **strictly decrease**	factorial(5) = **6** calls, O(n)
naive fib T(n) = **2F(n+1) − 1**	T(5) = **15**, O(φ^n), φ ≈ 1.618
memoise fib ⇒ **O(n)**	tail call ⇒ O(1) stack
Tree: n vertices ⇒ **n − 1** edges (induction)	Halting undecidable (contradiction)

Top 12 one-liners for MCQs

1. Induction needs **both** base case and step
2. The step is for **every** k, not one k
3. Contradiction assumes the **negation** of the goal
4. Contrapositive is **equivalent**; converse is not
5. **0! = 1** is the base case
6. Recursion needs a **smaller** argument
7. Naive fib is **exponential**, factorial is **linear**
8. A counterexample **refutes**, never proves
9. Finishing line: **"by the principle of induction"**

Top traps: checking the inductive step at one value of k · writing **P(k+1) ⇒ P(k)** instead · starting the base case at n = 1 when the claim says n ≥ 0 · assuming P instead of **¬P** in a contradiction proof · calling the contrapositive the converse · treating recursion as a proof rather than a definition · claiming naive factorial is slow · a recursive definition with no base case · "it held for the first few n, so it always holds".

All tricks together: "**B**ase **B**uilds, the **S**tep carries it **every** time" · "**A**stract the **p**ositive: assume the **n**egation and push it to the absurd" · "recursion must be **smaller**, or it is not recursion at all" · "the **contra**positive is the twin; the **con**verse is the liar" · "one counterexample ends the argument" · "fib **branches**, factorial **stretches**" · "correctness by induction, termination by shrinking".



one sentence decides which of the two proof techniques the question wants

✓ Practice MCQs (exam pattern, 10 marks)

1. A proof by contradiction begins by assuming:
   (a) the statement is true (b) the converse
   (c) the base case P(1) (d) the negation
2. In induction on P(n), the step must show:
   (a) for every k ≥ 1, P(k) ⇒ P(k + 1) (b) P(1) is true
   (c) P(k + 1) for one particular k (d) P(k + 1) ⇒ P(k)
3. a(1) = 1, a(n) = 2a(n−1) + 1. What is a(5)?
   (a) 31 (b) 15
   (c) 32 (d) 63
4. Which is equivalent to P → Q?
   (a) Q → P (b) ¬P → ¬Q
   (c) ¬Q → ¬P (d) P ∧ Q
5. $1 + 2 + 2^2 + \ldots + 2^n$ at n = 0 equals:
   (a) 0 (b) 1
   (c) 2 (d) 3
6. 0! equals:
   (a) undefined (b) 1
   (c) 0 (d) 2
7. Calls made by naive fib(5):
   (a) 5 (b) 11
   (c) 15 (d) 25
8. Calls made by naive factorial(5):
   (a) 5 (b) 6
   (c) 25 (d) 120
9. A single n where a claimed identity fails is:
   (a) an induction step (b) a contradiction
   (c) a counterexample (d) a base case
10. Always required of a recursive definition:
    (a) it must use a loop (b) it calls itself on a smaller argument
    (c) it must run in O(n) time (d) it must have a closed form

Answer key → 1-d · 2-a · 3-a · 4-c · 5-b · 6-b · 7-c · 8-b · 9-c · 10-b

Written question: (i) State the principle of mathematical induction and prove $1 + 2 + 2^2 + \dots + 2^n = 2^{n+1} - 1$ for every $n \geq 0$. (ii) Prove by contradiction that $\sqrt{2}$ is irrational. (iii) Give the recursive definition of the factorial function and the number of calls naive factorial(5) makes. (iv) Is the contrapositive of "n^2 even ⇒ n even" true? 10

1. **PMI** — P(1) true + P(k) ⇒ P(k+1) for every $k \geq 1$ ⇒ P(n) for all $n \geq 1$. **Base n = 0:** $2^0 = 1 = 2^1 - 1$ ✓. **Step:** from $1 + \dots + 2^k = 2^{k+1} - 1$, adding 2^{k+1} gives $2^{k+2} - 1$ ✓. 3
2. **√2** — suppose $\sqrt{2} = p/q$ in lowest terms; $p^2 = 2q^2 \Rightarrow p = 2k \Rightarrow q^2 = 2k^2 \Rightarrow$ q even.
   Both even ⇒ gcd(p, q) ≥ 2 ≠ 1 — contradiction. ■ 3
3. **Factorial** — n! = 1 if n = 0, else n · (n−1)!. Naive factorial(5) = **6** calls. 2
4. **Contrapositive** — "n^2 odd ⇒ n odd" is **true** and equivalent; the original is **false** (n = 3). 2

Before the exam ✓: write **both** halves of an induction proof · say "**for every k ≥ 1**" in the step · start a contradiction with the **negation** · name the principle in the closing line.